The Thiele Machine

A Complete Account from First Principles:
Structural Entitlement, Accountable Computation, and No Free Insight

Devon Thiele

April 2026

Abstract

I'm a car salesman. I started building this in January 2025 because I couldn't find a machine model that correctly accounts for the moment a computation becomes entitled to use structure. Fifteen months later, the result is machine-verified in Coq with zero admitted lemmas, implemented as RTL hardware, and executable in both extracted OCaml and Python. The thesis is not merely that proof costs something. The thesis is that ordinary computation is incomplete as a formal account if it models state evolution but not the acquisition of admissible structure: decompositions, morphisms, invariants, constraints, and certified narrowings that a later computation is allowed to use. No Free Insight is the first conservation law of that idea: any trace that starts uncertified and ends with a certified structural claim must have paid at least one unit in a global monotone ledger called μ. The structural-entitlement representation theorem closes the explicit witness case: strict narrowing plus distinguishability, certified posterior admissibility, and a decision-tree representative imply predicate strengthening, structure addition, and a $\Delta\mu$ lower bound. This document explains everything from first principles. What a Turing machine is and what it cannot see. What structural entitlement means. What insight means and why it cannot be free. The full 46-opcode instruction set. What a categorical morphism is, and why the morphism layer makes this machine strictly richer than any classical machine with the same registers and memory. The hardware design, which implements all 46 opcodes in RTL and has formal bisimulation proofs for all 46 of them (36 unconditional, 10 under inductive structural preconditions, zero gaps). The physics connections, which are carefully separated into what is proven and what is conditional. And how to break all of it.

Contents

1 Let me be straight with you

I'm a car salesman. I sell cars for a living.

I started building this in January 2025. I had no CS degree, no math degree, no physics degree. I barely knew what programming was. I used large language models as implementation tools: I specify what I want, the invariants it must satisfy, and the conditions under which it should fail, and I direct the AI to build it. Every Coq proof, every Python file, every line of RTL in this project was generated through that process. The ideas are mine. The proofs compile or they don't, and I verified every one of them.

I'm telling you this because it matters for how I think. I don't come from a background where anyone told me "this is how it works, trust us." I don't have professors or colleagues whose authority I can lean on. When I encounter a claim, I either find the proof or I don't believe it. That is not a methodology I adopted. It is how I think. It's the only way I know.

This document explains everything from the ground up. Not because I think you don't know it, but because I'm not confident I understand something until I can build it from scratch. So I explain it like I'm building it, because I am.

If you are an expert and you find this review of basics annoying, skip to Section 8. The technical content starts there. But if you find anything in the basics wrong, I want to know. Nothing here is correct just because I wrote it.

What this project is. The Thiele Machine is a model of computation where structural entitlement is a first-class state component. By entitlement I mean the machine is not merely storing data; it is allowed, by its own checked trace, to use a decomposition, relation, invariant, certificate, or narrowed feasible set as part of the computation. No Free Insight is the first theorem that falls out: a computation cannot end in a stronger certified claim than it started with unless the strengthening passed through the machine's explicit cost ledger. This is formalized in Coq and machine-checked. The ledger is called μ. It never goes down. If you go from uncertified to certified, μ went up. But I need to state the scope carefully: raw μ is a receipt that certification work was paid for. It is not, by itself, a universal depth score for the meaning of the claim. The stronger semantic reading only applies when the certification comes with an explicit narrowing witness. That distinction is not cosmetic. It is the hinge between an honest cost ledger and an overclaim.

What this project is not. It is not a claim that I solved P vs NP. It is not a theory of everything. It is not a claim that computation is physics or that physics is computation. The physics connections are real but conditional: they depend on named hypotheses that I label directly, and I am honest about which parts are proven and which are not. See Section 16.

2 The Turing Machine: what it can do and what it cannot see

Before I can explain what the Thiele Machine does differently, I have to explain what the Turing machine is and exactly where it falls short. Not vaguely falls short. Precisely.

2.1 What a Turing machine is

Alan Turing described his machine in 1936 to make precise the intuitive notion of "algorithm." The machine is this:

- An infinite tape. Think of it as an infinitely long strip of paper, divided into cells. Each cell holds one symbol from a finite alphabet, say $\{0, 1, \texttt{blank}\}$.
- A read/write head. It sits on one cell at a time. It can read the symbol in that cell, write a new symbol over it, and move one step left or one step right.

- A finite set of states. Call them $Q = \{q_0, q_1, \ldots, q_n\}$. The machine is always in exactly one state. q_0 is the start state. Some states are designated "halting" states.
- A transition table. For each combination of (current state, symbol under the head), the table says: write this symbol, move this direction, go to this new state.

That is the entire machine. Nothing else.

Here is the remarkable thing: this simple machine can compute *anything that can be computed.* Every algorithm that has ever been written, in every programming language that has ever existed, can be translated into a Turing machine. This is the Church-Turing thesis, and it has never been falsified. To falsify it, you would need to construct a physical device that reliably computes a function that no Turing machine can compute, not faster, but *at all.* No one has done it. Every physical computing device we have ever built turns out to be equivalent to a Turing machine in what it can compute. Your laptop, your phone, every server in every data center: all of them are, in the relevant theoretical sense, Turing machines. They can compute exactly the same class of things. The only difference is speed.

2.2 The blind spot

Here is the precise limitation. The transition table sees exactly two things: the current state q and the symbol under the head. That's it.

The machine cannot ask: "Is this tape sorted?" It has to read every cell.

The machine cannot ask: "Does this graph have two disconnected components?" It has to run a traversal.

The machine cannot ask: "Are these two parts of the input independent of each other?" It has to check.

The Turing machine's view of its tape is LOCAL. One cell at a time. It has no primitive way to see global structure. It can COMPUTE global structure, but computing it and seeing it are different things. To compute that a list is sorted, the machine has to execute a sorting-detection algorithm that touches every element. It pays in time and space for every structural fact it learns.

I spent months staring at this before it clicked. The Turing Machine isn't broken. It's blind by design. It's like trying to find your way through a maze by only ever looking at the floor tile you're standing on. You *can* do it. You will eventually reach the exit. But you're paying for every wrong turn, and you're paying for every part of the maze you already explored. Someone with a map doesn't pay that cost.

The Thiele Machine does not magically give the map. It represents the moment a computation becomes entitled to use a map, and it makes that entitlement auditable. The map is not free.

2.3 Classical complexity ignores structural cost

Standard complexity theory measures two things: time (number of steps) and space (number of cells or registers used). It says nothing about the cost of knowing that the input has structure.

Consider the satisfiability problem (SAT): given a logical formula ϕ over n boolean variables, is there an assignment that makes it true? In the worst case, a blind machine has to check all 2^n assignments. That's what makes SAT hard.

Now suppose ϕ decomposes cleanly: $\phi = \phi_1 \wedge \phi_2$, where ϕ_1 and ϕ_2 share no variables. You can solve ϕ_1 on its own (2^{n_1} work) and ϕ_2 on its own (2^{n_2} work). Total work: $2^{n_1} + 2^{n_2}$ instead of 2^n. That is an exponential improvement.

But that improvement only applies if you *know* the decomposition exists. And the Turing machine model assigns zero cost to knowing it. The structure is either there or it isn't, and you either see it or you spend time finding it, but the act of certifying "I know this decomposition is valid" is not tracked, not paid for, not audited.

The Thiele Machine says: that act belongs inside the computation. Here is the structural object. Here is the checked transition that makes the object admissible. Here is where the cost shows up in the trace. Here is how to verify that the cost was actually paid. This is the structural-entitlement reading of No Free Insight.

3 What structure is

"Structure" is one of those words everyone uses and nobody defines. I'm going to define it precisely because the whole machine rests on this definition.

3.1 Partitions

Start with the simplest case. Take any set S and divide it into labeled buckets that don't overlap and cover everything. That is a partition.

Formally: a partition of S is a collection of disjoint non-empty subsets $S_1, S_2, \ldots, S_k$ such that $S_1 \cup S_2 \cup \cdots \cup S_k = S$. Each S_i is called a module or block or cell, depending on the context.

Now here is the key thing: the partition IS information. Knowing that element x belongs to block S_1 and element y belongs to block S_2 tells you something real. It tells you x and y are in different groups. If you didn't know the partition, you didn't know that. If you know the partition, you can look it up in constant time.

In a computational context, partitioning the memory or register state means saying: "These addresses belong to module A, those addresses belong to module B, and these two modules are independent." That is structural knowledge. A Turing machine has no way to represent this directly. It has to recompute independence from scratch every time it needs to know it. The Thiele Machine stores the partition as part of its state, audits the cost of building it, and enforces that building it was paid for.

3.2 Axioms on modules

A partition alone is just a labeling. What makes it structural knowledge is when you attach *constraints* to the modules. For example: "Module A's memory region satisfies the predicate Φ_A," where Φ_A might say "this region is a valid binary heap" or "all values in this region are between 0 and 100" or "this region encodes a satisfying assignment to formula ϕ."

These constraints are called axioms. They are written in a formal logic. Concretely, the repository has two formula surfaces. The text-level certificate checker in `CertCheck.v` uses DIMACS CNF-style formulas with assignment/certificate text. The on-chip `LASSERT` path uses a binary memory layout: a formula header stores the encoded word count, variable count, and clause count, followed by literal words. The certificate block stores one satisfying assignment and one falsifying assignment. The Logic Engine finite-state machine reads those words from memory. If validation succeeds, the checked constraint package can be used by the later certification path. If validation fails, the machine traps and sets an error flag.

The crucial point: you cannot add an axiom to a module for free. Adding an axiom means the machine paid to certify it. The axiom is evidence, not assertion.

3.3 Structural gain

A trace has "structural gain" if it starts in a state where some structural fact is not certified and ends in a state where it is. The classic example: a trace that starts with an uncertified partition and ends with a certified satisfying-assignment witness has undergone structural gain.

The Thiele Machine's core question is: where, in the trace, did the structural gain enter? Not merely "did the gain happen." That is visible from inspecting the start and end states. The question is which instruction, at which step, introduced the certified structural knowledge.

The answer is always: an instruction that costs $\mu \geq 1$. That is No Free Insight.

4 Structural entitlement

This is the part I need to say more plainly. The center of the project is not just "certification has a price." That is true, but it is the first conservation law, not the whole picture.

The deeper object is *structural entitlement*. A computation has structural entitlement when the state does not merely contain data, but contains an admissible structural fact that later steps are allowed to use: a partition, a decomposition, a morphism, a constraint, a witness, or a narrowed feasible set. The difference matters. A raw bit pattern can be copied. A certified decomposition can justify solving two smaller problems instead of one large one. A raw morphism ID can be forged in a register. An asserted morphism in the graph has provenance. A formula in memory is text. A checked formula with a model and a countermodel is admissible narrowing.

Definition 1 (Structural entitlement)**.** *A state has structural entitlement to a claim P when three things are present:*

- *a structural object in the machine state, such as a partition, morphism, witness counter, or formula package;*
- *a checked relation between that object and the claim P;*
- *a trace-visible certification or assertion event that makes later use of P admissible.*

Without the third part, the object may exist as raw data, but the computation has not earned the right to use it as certified structure.

The repository proves the pieces of this idea and now proves the weld between them.

Theorem 1 (Classical blindness as quotient, `BlindnessRepresentation.v` and `ShadowProjection.v`, zero Admitted)**.** *The four-field forgetful map from Thiele states to Turing-style snapshots is a many-to-one quotient. Its kernel is exactly equality on PC, μ, registers, and memory, so it can identify states that differ in certification, witness counters, or graph structure. The six-field `shadow_proj` keeps `vm_certified` but still forgets morphism structure; no function of that shadow alone can separate the morphism witness states.*

Plain version: if your machine state only records registers, memory, PC, error bit, and maybe μ, you have already thrown away part of what matters. You have collapsed different epistemic states into one raw state. That is not a philosophical complaint. It is a theorem about a projection map.

Theorem 2 (Feasible narrowing becomes predicate strengthening, `InformationGainToStrengthening.v`, zero Admitted)**.** *If a posterior feasible set Ω' is a strict subset of a prior feasible set Ω, and there is an observation that distinguishes a removed witness state from all posterior states, then the membership predicate for Ω' is strictly stronger than the membership predicate for Ω.*

This is the bridge from "the search space got smaller" to "the computation now accepts a stronger claim." The predicates are not fake constants. They are built from membership in the feasible sets through an observation function.

Theorem 3 (Strengthening requires structure addition, `NoFreeInsight.v`, zero Admitted)**.** *If a trace starts with no certification address, ends certified for a strictly stronger predicate, and the certification is tied to the machine's supra-certification channel, then the run contains a structure-addition event.*

This is the formal answer to "where did the new admissible structure enter?" It did not appear from arithmetic alone. It entered through a trace-visible structure-addition event.

Theorem 4 (Universal certification cost, `UniversalCertificationCost.v`, zero Admitted)**.** *For any abstract certification system with a state type, instruction type, step function, cost function, and certification predicate, if every single-step transition from uncertified to certified costs at least 1, then every trace from uncertified to certified has total cost at least 1.*

This last theorem is important because it is not about the Thiele ISA. It says the conservation law is forced for any system once the system admits a real certification predicate and a sound cost rule for certification transitions. If a model lets the certification predicate flip at zero cost, then it has free certification by construction. If it refuses to represent certification at all, then it cannot talk about structural entitlement internally. It can only smuggle that entitlement in from outside the model.

Theorem 5 (Structural entitlement representation, `HonestNoFI_TheoremsWithoutAssumptions.v`, zero Admitted)**.** *For any explicit witness of structural entitlement consisting of:*

- *a strict feasible-set narrowing $\Omega' \subsetneq \Omega$;*
- *an observation function that distinguishes at least one eliminated witness state from all posterior states;*
- *a certified posterior predicate built from membership in Ω';*
- *a decision tree realized by the trace; and*
- *a posterior-representative reduction tying the prior states to posterior representatives,*

the posterior predicate is strictly stronger than the prior predicate, the trace contains a structure-addition event, and the quantitative narrowing is bounded by the paid ledger:

$$\log_2^{\uparrow} |\Omega| - \log_2^{\uparrow} |\Omega'| \leq \Delta\mu.$$

This is the theorem I needed in the neighborhood of the deeper thesis. It does not quantify over the English word "structure." It defines the witness class: strict narrowing, distinguishability, certification, and an explicit tree/fiber representative for the reduction. For every member of that class, structural entitlement is not metadata and not a slogan. It is predicate strengthening, it requires structure addition, and its quantified narrowing is charged to μ.

Theorem 6 (Every sound structural shortcut lands here, `HonestNoFI_TheoremsWithoutAssumptions.v`, zero Admitted)**.** *A `SoundStructuralShortcut` is a package containing exactly the receipts a shortcut must provide to count as sound: a prior feasible set, a posterior feasible set, strict narrowing, a distinguishing observation, a certified posterior predicate, a realized decision tree, and a posterior-representative reduction. For every such package, the structural-entitlement representation theorem applies. The shortcut lands in strictly stronger posterior knowledge, a structure-addition event, and the $\Delta\mu$ lower bound.*

This is the formal version of "it has to." Once a shortcut is no longer just a human story but a sound object the machine may use, it has to provide the data that makes the shortcut admissible. In this framework, that data is exactly the structural-entitlement witness. If someone says symmetry, factorization, modularity, independence, or decomposition gave them the shortcut, the next question is no longer mystical. Show me the prior set, the posterior set, the observation that rules something out, and the representative/tree witness. If they can show that, the theorem applies. If they cannot, then the structure is still outside the computation.

4.1 What narrowing buys

The machine also has a concrete structural-advantage demonstration. `StructuralAdvantage.v` studies a factored search problem. The blind program has all instruction costs set to 0 and searches the flat space. The sighted program pays two receipt-visible events, each `EMIT "." 0`, for total cost 18, and searches two factors separately. The file proves the exact cost formulas and the arithmetic fact that the N^2 versus $2N$ gap eventually dwarfs the constant μ cost. The executable tests check the measured VM behavior on representative sizes.

The repository now separates two nearby claims instead of blurring them. The original `EMIT`-only $N = 1$ witness closes the observation layer: it proves posterior admissibility as `CertifiedObs`. It does not close the stronger bridge. The run does not end in `has_supra_cert`, and in the semantic CSR-transition sense it does not realize structure addition. A minimally extended witness does close the full theorem honestly: keep the factorized search exactly as it is, then append `PNEW`, `MORPH_ID`, and `MORPH_ASSERT`. That suffix is enough to fire the concrete certification channel, so the extended trace lands in the full structure-addition theorem.

That example is not a P vs NP result. It is a small, honest witness for the thing this machine is meant to track: certified structure can change which computation is admissible, and the cost of acquiring that admissibility is not the same as the time saved by using it.

4.2 The boundary

What I still do not have is the open translation theorem: whether every human informal story about symmetry, modularity, factorization, independence, or decomposition can always be packaged as a `SoundStructuralShortcut`. That would be a full informal-to-formal representation theorem for structural use, and it is stronger than what the repository proves today. It also sets the standard for the work: if a claimed shortcut can be pushed down into explicit witnesses, it has to be pushed to bedrock. If it can still be pushed, push again. Stop only where the proof honestly stops.

The repository now proves the hard internal theorem for the explicit class, and it proves the boundary in both directions on a nearby concrete witness. The original `EMIT`-only factorized-search trace closes only the observation layer: the posterior predicate is certified observationally, but the run does not reach `has_supra_cert` and does not realize semantic structure addition. A minimally extended trace, with the factorized search unchanged and only a post-search suffix `PNEW ; MORPH_ID ; MORPH_ASSERT`, does close the whole chain: stronger predicate, structure addition, and the $\Delta\mu$ lower bound. The unconditional Shannon-style statement for every deterministic single trace is still explicitly rejected in the code. The bridge needs a whole-tree, fiber, or distributional witness. That is not a weakness in the proof. It is where the proof stopped lying.

So the honest thesis is this: once structural entitlement is internalized with explicit witnesses, free certified strengthening is impossible; classical shadows lose entitlement-relevant distinctions; and feasible-set narrowing has a formal path into predicate strengthening and a ledger lower bound. The broader project is to show how many real structural shortcuts can be compiled into those witnesses.

5 What insight is, and why it cannot be free

In this machine, insight has a precise meaning. Now that structural entitlement is on the table, I can define the different kinds of observation the machine allows.

5.1 The three tiers of observation

Not all observations are the same. The taxonomy is proven in Coq. File: `WitnessInsightGeneral.v`.

Tier 0: Raw observation (always free). A raw observation records data without committing to any structural interpretation. The clearest example in the Thiele Machine is a CHSH trial (see Section 14): you run an experiment, you record the outcome in a counter, and nothing structural changes. The counter goes up. The cert channel is not touched. This is free. Cost: 0.

Tier 1: Certified structural insight (always costs ≥ 1). The machine has two recorded certification channels. One is the formula/certificate address channel, `csr_cert_addr`. The other is the top-level flag, `vm_certified`. Any instruction that turns either channel from "off" to "on" must have cost ≥ 1. This is proven in Coq (`certification_requires_positive_mu`). Zero Admitted. I need one precision point here: in the current hardware-aligned step relation, `LASSERT` checks a constraint package and charges for it, but the concrete state-changing channels are `CERTIFY` for `vm_certified` and `MORPH_ASSERT` for `csr_cert_addr`. `AbstractNoFI.v` also reasons about the broader certification/revelation class as a positive-cost policy.

Tier 2: Certified non-local witness insight (always costs ≥ 1). The sharpest case: a machine that is certified AND whose CHSH statistics show a violation of the classical bound ($S > 2$; more on CHSH in Section 14). Getting certified is already a Tier 1 event (cost ≥ 1). Getting the statistics to show a violation means the data can't be explained by any shared random strategy; this is proven in `CHSHStatisticalBridge.v`. Together: certified non-local evidence costs at least 1 to acquire, over any trace of any length.

5.2 Why insight cannot be free

Here is the argument. It is a proof, not an intuition.

Suppose a machine starts uncertified and ends certified. Somewhere in that trace, a specific opcode fired that flipped one of the certification registers from off to on. Either CERTIFY ran and set the top-level certified flag to true, or MORPH_ASSERT ran and proved a morphism exists. Look at the exact step where that happened. That step, by the semantics of the machine, has cost ≥ 1. And because μ never decreases (proven in `MuLedgerConservation.v`), the final μ is at least $\mu_{\text{before that step}} + 1 \geq \mu_{\text{initial}} + 1$.

This is not a philosophical argument. It is a structural property of the step function. The Coq proof checks it mechanically. The proof does not say "intuitively, certification should cost something." It says: here is the step function definition, here is the cost semantics, here is the monotonicity lemma, here is the induction on the trace. QED. Zero Admitted.

The deeper reason is this: if you could certify structure for free, the certification would be meaningless. A receipt that any machine can produce at zero cost proves nothing. The whole point of the μ ledger is that it is unforgeable. You cannot accumulate certified structural knowledge without a trace of cost.

But cost alone is not yet semantic depth. A machine can spend resources checking a claim that turns out to be shallow. So in the tightened architecture, LASSERT does not count as structural certification merely because a formula parses and one satisfying assignment exists. It must also carry a non-triviality witness: one satisfying assignment and one falsifying assignment. That proves the asserted constraint excludes at least one state and therefore genuinely narrows the feasible set. The receipt proves spend; the witness proves the spend bought an actual narrowing.

6 The μ ledger

μ (mu) is the global structural cost ledger of the Thiele Machine. Let me explain what it is from first principles.

6.1 What mu is

μ is a natural number attached to every machine state. It starts at 0. It never decreases.

Every instruction carries a cost parameter (δ) in its encoding. The machine adds that cost to μ when the instruction executes. Here is how δ is used in practice:

For **pure compute programs**, meaning programs that do arithmetic, load and store values, and jump, every instruction carries $\delta = 0$. The program I use as the concrete comparison case for the Turing Strictness theorem (`blind_program` in `StructuralAdvantage.v`) has this property: all instructions at $\delta = 0$, and the proof that its total μ-cost is zero is a direct calculation. For these programs, μ stays at 0 unless a positive-cost instruction fires.

For **certification and revelation instructions** (CERTIFY, LASSERT, MORPH_ASSERT, EMIT, REVEAL, LJOIN, READ_PORT): the machine applies a mandatory wrapper regardless of δ. Even with $\delta = 0$, the cost is at least 1. This is the enforcement that makes No Free Insight work. A real program in `StructuralAdvantage.v` uses `EMIT` with the payload "." and $\delta = 0$; the proof unfolds that payload into eight concrete bits and confirms it costs exactly 9 ($|\text{payload}|_{\text{bits}} + S(0) = 8 + 1 = 9$). Throughout this document, $S(n)$ means $n + 1$. It is Coq's Peano successor function. I use it because the formal proofs are written in Coq and I want the formulas here to match the actual proof terms. $S(\delta) = \delta + 1 \geq 1$ always, which is precisely the enforcement that this class can never be free even at $\delta = 0$. EMIT, REVEAL, and READ_PORT go further than the simple $S(\delta)$ floor: their cost is proportional to the content they carry in actual bits, so certifying more information costs more μ than certifying less.

For **everything outside that positive-cost class**, including ADD, LOAD, STORE, JUMP, PNEW, PSPLIT, PMERGE, MORPH, COMPOSE, and the rest, the machine charges exactly δ. These operations can be structurally free when $\delta = 0$. Creating a partition module, building a morphism chain, doing arithmetic: none of that is forced to accumulate μ. The design principle in `MuCostDerivation.v` (`partition_ops_cannot_cost`) makes the intended policy explicit: reversible partition operations have zero information cost, so their canonical cost is 0.

μ grows whenever `instruction_cost` is positive. The certification/revelation class is forced to be positive even when its encoded δ is 0. Other instructions can remain free by carrying $\delta = 0$.

The proof that μ is always exactly μ_{before} + `instruction_cost` for each step (where `instruction_cost` is δ for ordinary instructions; $S(\delta)$ for CERTIFY, MORPH_ASSERT, and LJOIN; $|\text{payload}|_{\text{bits}} + S(\delta)$ for EMIT; bits $+ S(\delta)$ for REVEAL and READ_PORT; and *flen* $\times 8 + S(\delta)$ for LASSERT):

Theorem 7 (μ-Conservation, `MuLedgerConservation.v`, zero Admitted)**.** *For any state s and instruction i:* (*vm_apply* s i).μ = $s.\mu$ + *instruction_cost* i.

This is not a monotonicity claim. It is an equality. μ goes up by exactly the cost of the instruction, always, for every instruction, in every case including error cases. There is no backdoor.

That equality should not be misunderstood. It means the ledger is exact about spend. It does not, by itself, mean every paid unit corresponds to deep semantic gain. In this machine, semantic gain is stricter than spend: it requires a narrowing witness in addition to the ledger entry.

6.2 Why mu is the unique canonical cost measure

Here is the stronger claim, which took me a while to prove: μ is not just A valid cost measure. It is THE cost measure. Any other non-decreasing cost functional that starts at zero and is consistent with the instruction step semantics must be isomorphic to μ.

Theorem 8 (μ-Initiality, `MuInitiality.v`, zero Admitted)**.** *Let M be any function on machine states satisfying: (a) $M(s_{init}) = 0$ for all initial states, (b) $M(s') \geq M(s)$ for all steps $s \to s'$, and (c) $M(\texttt{vm\_apply}\ s\ i) = M(s) + \texttt{instruction\_cost}\ i$ for all reachable states. Then $M = \mu$ on all reachable states.*

In plain language: there is no alternative accounting. If you want to track structural cost on this machine, you end up at μ. The cost structure was forced by the machine design, not chosen arbitrarily.

6.3 The LASSERT cost formula

The most important cost formula in the machine is for LASSERT, the opcode that certifies a logical formula. Let me build it up:

$$\Delta\mu_{\text{LASSERT}} = \mathit{flen} \times 8 + S(mu\_delta)$$

flen is the encoded formula-unit count, read from the formula's own header in memory. Each unit contributes eight concrete bits, and the per-step certification charge is always at least 1.

Plain version: the machine reads how many encoded formula units are actually present, charges eight bits for each unit, and adds at least 1 for the certification step. A longer formula costs more. A two-unit formula costs at least $16 + 1 = 17$. The programmer does not get to smuggle a larger formula through a smaller bit count.

This is forced, not chosen. You cannot certify a 1000-bit formula by paying for a 10-bit one; the cost counts the formula you actually present. The proof that this is the unique minimum cost satisfying that constraint is in `MuCostDerivation.v` (`cost_necessity` + `cost_uniqueness`), zero Admitted.

I need to be honest about what this buys you, though. The formula pays for presenting and checking a constraint package. It does not, by itself, guarantee the constraint is meaningful. You could present a formula that is always true, a tautology, and the ledger would still charge you. The machine closes this separately via the dual-witness requirement: see the LASSERT opcode description in Section 9.

The natural attack, and why it doesn't work. The programmer declares *flen* in the instruction encoding. What stops them from writing $\mathit{flen} = 0$ for a huge formula and paying only $S(0) = 1$?

The hardware catches it. When LASSERT runs, the machine reads the formula's actual encoded-unit count from the formula's own header in memory and compares it to the *flen* in the instruction. If they don't match, the machine traps: the program counter moves to `LASSERT_TRAP_PC`, the error flag is set, and the check does not succeed. The μ cost is still charged. That detail matters. Failed checks are not free. This is proven in `StateSpaceCounting.v` (`lassert_honest_cost` and `lassert_honest_mu_cost`, zero Admitted): if LASSERT completes without trapping, the declared count equals the in-memory count, and the μ charge uses the real formula bits.

7 No Free Insight: the first conservation law

Now I can state the first conservation law precisely.

7.1 The abstract versions

Theorem 9 (Universal No Free Certification, `UniversalCertificationCost.v`, zero Admitted)**.** *Let $\mathcal{C}$ be any certification system with:*

- *a state type;*
- *an instruction type;*
- *a step function;*
- *a cost function;*
- *a certification predicate.*

If every single-step transition from uncertified to certified has cost ≥ 1, then any trace that starts uncertified and ends certified has total trace cost ≥ 1.

This theorem is substrate-independent. It does not know about registers, memory, partitions, morphisms, CHSH trials, or the Thiele ISA. It says: once a model has a real certification predicate and the one-step certification transition is priced, trace-level non-free certification follows by induction. If that one-step premise fails, the model permits free certification. If the model has no certification predicate, it cannot state structural entitlement internally.

Theorem 10 (Thiele No Free Insight, `AbstractNoFI.v` and `NoFreeInsight.v`, zero Admitted)**.** *For the Thiele Machine specifically, cert-channel activation and certified predicate strengthening require structure-addition events, and those events are in the positive-cost certification/revelation class.*

This is the machine-level version. `AbstractNoFI.v` proves the two concrete cert channels cannot be activated for free. `NoFreeInsight.v` adds the predicate-strengthening layer: once a stronger accepted predicate is tied to `has_supra_cert`, the run must contain a structure-addition event.

7.2 The specific versions

For the Thiele Machine concretely, there are several specific versions. The two certification channels I mentioned in Section 5 are the formula-certification register (`csr_cert_addr`) and the top-level flag (`vm_certified`). Both are defined precisely in Section 8.

Theorem 11 (No Free Certification, single step, `AbstractNoFI.v §8`, zero Admitted)**.** *If the formula-certification register goes from absent to present in a single step, the instruction that caused it cost ≥ 1.*

Theorem 12 (No Free Certification, trace level, `AbstractNoFI.v §9`, zero Admitted)**.** *If the formula-certification register was absent at the start of a trace and is present at the end, then the trace paid at least 1 in μ.*

Theorem 13 (No Free Certification via CERTIFY, `AbstractNoFI.v §10`, zero Admitted)**.** *If the top-level certified flag goes from false to true in a single step, the instruction that caused it cost ≥ 1.*

Theorem 14 (Both channels, `AbstractNoFI.v §11`, zero Admitted)**.** *For ANY transition of either certification channel from absent to present, whether the formula register or the certified flag, μ grows by at least 1.*

Theorem 14 is the master NoFI result. It covers both ways to get certified. There is no third way.

7.3 What this means in practice

Run any program on the Thiele Machine. Watch the μ counter. If the program ends with `vm_certified = true` or with `csr_cert_addr` nonzero, then somewhere in the execution, μ went up by at least 1. You can audit the trace and find exactly where. There is no program that can reach a certified state from scratch without paying for it.

This is what I mean by unforgeable. The μ ledger is the receipt. If you are certified, the receipt exists.

8 The formal machine state

Let me now define the Thiele Machine precisely. I will go through every field.

Definition 2 (Thiele Machine State)**.** *A Thiele Machine state s is a record with 12 fields:*

- *s.`vm_graph`: the partition graph. It contains module records, morphism records, and fresh-ID counters.*
- *s.`vm_csrs`: control-status registers. This includes `csr_cert_addr`, where 0 means no formula/certificate address is present.*
- *s.`vm_regs`: a list of register values. The kernel fixes `REG_COUNT = 32`; register reads and writes wrap indices modulo 32.*
- *s.`vm_mem`: a list of data-memory words. The kernel fixes `MEM_SIZE = 65536`; memory reads and writes wrap indices modulo 65536.*
- *s.`vm_pc` $\in \mathbb{N}$: the program counter.*
- *s.`vm_mu` $\in \mathbb{N}$: the structural cost ledger. Starts at 0. Never decreases.*
- *s.`vm_mu_tensor`: a flattened 4 by 4 cost tensor, 16 entries in the kernel model.*
- *s.`vm_err` $\in \{false, true\}$: the error flag. Set when an instruction fails.*
- *s.`vm_logic_acc`: logic-engine accumulator state.*
- *s.`vm_mstatus`: machine-status word.*
- *s.`vm_witness`: the CHSH witness counters.*
- *s.`vm_certified` $\in \{false, true\}$: the top-level certification flag. Set by the CERTIFY opcode.*

The kernel and RTL are intentionally not the same shape at every bit. The Coq kernel stores register and memory values as natural numbers but truncates register and memory writes through its `word64` helpers. The Kami RTL target is finite hardware: 32-bit data registers, 65536 memory words, bounded partition and morphism tables, and 128-bit instruction transport. The bridge proofs state exactly where those finite hardware representations match the kernel step.

Definition 3 (Partition Graph)**.** *The partition graph s.`vm_graph` is a record:*

- *`pg_next_id`: the next fresh module identifier.*
- *`pg_modules`: a list of module-ID/module-record pairs. Each `ModuleState` holds region data, axiom data, and the module's μ-tensor.*
- *`pg_next_morph_id`: the next fresh morphism identifier.*

- *pg_morphisms: a list of morphism-ID/morphism-record pairs. Each* `MorphismState` *holds a source module, target module, coupling data, and an identity flag.*

Remark 1. *There is no separate* `vm_heap`*,* `vm_output`*, or* `vm_imem` *field in the Coq kernel's* `VMState`*. Heap operations are heap-relative accesses into* `vm_mem`*. Output and instruction transport live at the runner, trace, or RTL harness layer, not as kernel state fields. This distinction matters because NoFI is a theorem about the state record above.*

Remark 2. *The partition graph has two distinct layers. The* module layer *records how the state is decomposed into named pieces. The* morphism layer *records typed categorical relationships between those pieces. These layers are independent. Two machine states can agree on every module while differing in their morphism maps. This independence is what makes the Categorical Separation Theorem possible (Section 11).*

9 The instruction set: all 46 opcodes

The Thiele Machine has 46 opcodes. Every one carries an explicit `mu_delta` parameter. Cost is part of the instruction, not an annotation. I will explain each group from first principles.

9.1 Group 1: Partition operations (4 opcodes)

These opcodes modify the partition graph. They are how the machine builds structural knowledge.

PNEW(R, δ): allocate a new module.
Creates a fresh module and charges δ to μ. The instruction carries a region list R, but the current hardware-aligned semantics use the length of the normalized region and store the canonical region `seq 0 sz`. So the checked step preserves the module size, not arbitrary input geometry. The bounded RTL has its own partition-table and active-module machinery for locality enforcement, but `PNEW` itself is the graph allocation step.

Why does this exist? Because structural knowledge needs a home. Creating a module is how you tell the machine "this region of computation is a named unit I am tracking." The cost δ is programmer-declared and can be 0; the machine does not force you to pay for allocation itself. If building this partition cost something because you had to figure out the right boundary, run a search, or derive the structure, you record that here. If it was bookkeeping, $\delta = 0$ is fine.

PSPLIT(m, L, R, δ): split one module into two.
Splits module m and charges δ to μ. The constructor still carries left and right region payloads, but the current hardware-aligned step ignores those payloads and calls `graph_hw_psplit`: the left side gets half the module size, and the right side gets the remainder. This is the refinement operation in the finite hardware model. If discovering the split deserves a cost, encode a positive δ; the canonical reversible-accounting policy keeps it at 0.

PMERGE(m_1, m_2, δ): merge two modules into one.
Merges modules m_1 and m_2 and charges δ. The hardware-aligned step concatenates the module sizes through `graph_hw_pmerge`; module IDs wrap modulo 64. It does not perform a rich invariant-compatibility check in this step. Like PNEW and PSPLIT, the machine does not enforce a minimum cost here. The mandatory cost comes later if you try to certify a claim about the structure.

PDISCOVER(m, ***evidence***, δ): query partition structure.
The evidence payload is carried by the instruction, but the hardware-aligned relation does not attach axioms or mutate the graph. It advances the PC and charges δ. This is a placeholder for discovery accounting, not a register query in the current kernel step.

9.2 Group 2: Logic and certification (3 opcodes)

These opcodes interact with the Logic Engine, the on-chip finite-state machine that verifies formulas and certificates.

LASSERT(*freg*, *creg*, *kind*, *flen*, *cost*): assert and certify a formula.
Reads a formula whose declared encoded-word count is *flen* from memory starting at the address in register *freg*. The *kind* boolean flag selects the certificate path expected by the Logic Engine FSM. In the current on-chip semantics, *kind* = *true* is the SAT path and *kind* = *false* (UNSAT) always fails. The kernel and hardware both validate that the declared count matches the in-memory formula header before the check is allowed to succeed. A mismatch traps and sets the error flag. Reads a certificate block from memory starting at the address in register *creg*. In the tightened architecture, that block contains two witnesses: one satisfying assignment and one falsifying assignment. The on-chip Logic Engine FSM processes the formula and both witnesses in-place without external solver calls. Validation succeeds only if the declared count is honest, the first witness satisfies the formula, and the second witness falsifies it. That means the formula is not a tautology and really excludes at least one state. LASSERT itself is a checked validation step; certification channels are activated by the concrete state-changing paths such as CERTIFY and MORPH_ASSERT, while the abstract cert-address theorem treats LASSERT as part of the certification/revelation class. Charges $\mathit{flen} \times 8 + S(\mathit{cost})$ to μ, even when the check fails.

This is the most important logic opcode in the machine. It is where a constraint package is checked before any later certification step can lean on it. You cannot fake the pricing anymore by declaring a tiny *flen* for a large in-memory formula: if the header and the instruction disagree, the machine traps. You also cannot fake semantic narrowing with a tautology: the dual-witness requirement closes the tautology-inflation hole because there is no falsifying witness to show the formula excluded any state. The cost formula is still a spend ledger. The honest-length check and the non-triviality witness are what tie that spend to the real checked object.

LJOIN(*c1reg*, *c2reg*, δ): join two certificates.
Combines two memory-resident certificates into a single composite certificate. The addresses of the input certificates are in *c1reg* and *c2reg*. Charges $S(\delta) \geq 1$. This supports constructing complex multi-part proofs from simpler pieces.

MDLACC(m, δ): MDL cost accounting.
Charges δ as a module-discovery/model-description cost for module m. It does not read a register, and it does not mutate the graph.

Here is why this opcode exists. Suppose you have two models that both fit the same data. One model is a ten-thousand-parameter neural network. The other is a five-line formula. Both fit the training set. Which one are you buying? The minimum-description-length principle says: the total bill is the model description itself plus the cost of encoding the data once you have the model. The short formula costs more to encode data with, but costs almost nothing to describe. The massive network achieves perfect in-sample compression, but its own description is enormous. The correct choice minimizes the sum.

Translated to the Thiele Machine vocabulary: a module structure that you built has a description-length cost. Not because someone told you it does, but because constructing a specific partition of state space may require information that should be billed. MDLACC is the opcode that lets a program explicitly enter that description cost into the ledger after the structure has been built. It performs no graph change. Its sole effect is to advance the program counter and charge the encoded δ. The connection to MDL is not a citation to a 1978 paper; it is the claim that the machine's natural μ-accounting can track what information-theoretic compression theory says a model costs, when the program chooses to enter that cost.

9.3 Group 3: Compute and data transfer (14 opcodes)

Standard arithmetic and data movement. These are the operations you find in every ISA. They mostly carry $\delta = 0$ because they do not introduce new structural knowledge. They just compute.

Opcode	What it does	Typical cost
`LOAD_IMM`(r, v, δ)	Write immediate value v into register r	0
`XFER`(r_{dst}, r_{src}, δ)	Copy register r_{src} to r_{dst}	0
`ADD`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a + r_b$ (modular 64-bit)	0
`SUB`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a - r_b$ (modular 64-bit)	0
`MUL`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a \times r_b$	0
`AND`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ AND r_b (bitwise)	0
`OR`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ OR r_b (bitwise)	0
`SHL`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted left by r_b bits	0
`SHR`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted right by r_b bits	0
`LUI`(r_d, v, δ)	Load upper immediate: $r_d \leftarrow v \ll 8$	0
`XOR_LOAD`(r_d, $addr$, δ)	Load from absolute memory address $addr$ into r_d	0
`XOR_ADD`(r_d, r_s, δ)	$r_d \leftarrow r_d$ XOR r_s (XOR accumulate)	0
`XOR_SWAP`(r_a, r_b, δ)	Swap r_a and r_b via XOR (reversible)	0 by convention
`XOR_RANK`(r_d, r_s, δ)	$r_d \leftarrow$ popcount(r_s) (Hamming weight)	0

Why does the ISA have all these? Because the Thiele Machine is a complete computing machine, not just a structural accounting system. Programs need to compute intermediate values, manage data, and control flow. The compute group provides all of that. The key property: none of these opcodes touch the certification channels. They cannot create certified structural knowledge.

XOR_SWAP is specifically reversible. It uses the XOR trick to swap two registers without a temporary, and it carries $\delta = 0$ by convention because reversible operations do not increase information entropy.

9.4 Group 4: Memory and control flow (8 opcodes)

Memory access and program flow control.

Opcode	What it does	Note
`LOAD`(r_d, r_a, δ)	Load `vm_mem`[r_a] into r_d	Kernel register-indirect load
`STORE`(r_a, r_s, δ)	Store register r_s into `vm_mem`[r_a]	Kernel register-indirect store
`JUMP`(addr, δ)	Set PC to addr unconditionally	none
`JNEZ`(r, addr, δ)	Jump to addr if $r \neq 0$	none
`CALL`(addr, δ)	Push return address via stack pointer r31, jump to addr	Qed coverage
`RET`(δ)	Pop return address via stack pointer r31, jump there	Qed coverage
`HALT`(δ)	Stop execution	none
`CHECKPOINT`(ℓ, δ)	Accept checkpoint label ℓ and advance	label handled outside kernel state

At the kernel level, LOAD and STORE are register-indirect memory operations over `vm_mem`. At the RTL/cosimulation layer, the partition wall adds active-module locality checks. `INIT_PT`

and `INIT_ACTIVE_MODULE` are harness setup commands for that finite hardware table, not VM opcodes and not fields of `VMState`. This is the honest split: the abstract ISA defines memory semantics; the bounded hardware layer enforces locality through its own table state.

9.5 Group 5: I/O, heap, and tensor (7 opcodes)

Input/output and non-register storage.

Opcode	What it does	Cost
`READ_PORT`(r, c, v, *bits*, δ)	Write decoded input value v from channel c into register r	$bits + S(\delta) \geq 1$
`WRITE_PORT`(c, r_s, δ)	Send register r_s to channel c outside kernel state	δ
`EMIT`(m, *payload*, δ)	Emit payload as a certified trace event	$\lvert payload\rvert_{\text{bits}} + S(\delta) \geq 1$
`HEAP_LOAD`(r_d, r_a, δ)	Load from `csr_heap_base + regs[`r_a`]` into r_d	δ
`HEAP_STORE`(r_a, r_s, δ)	Store r_s to `csr_heap_base + regs[`r_a`]`	δ
`TENSOR_SET`(m, i, j, v, δ)	Set per-module tensor entry (i, j)	δ
`TENSOR_GET`(r_d, m, i, j, δ)	Read per-module tensor entry (i, j) into r_d	δ

EMIT and READ_PORT are in the certification/revelation cost class, but their cost goes beyond the simple $S(\delta)$ floor. EMIT charges $|\text{payload}|_{\text{bits}} + S(\delta)$: the machine unfolds the emitted payload into concrete bits and charges for those bits directly, then adds at least 1 for the certification step. A one-byte ASCII payload has 8 payload bits, so with $\delta = 0$ it costs 9. A ten-byte ASCII payload has 80 payload bits, so with $\delta = 0$ it costs 81. The programmer cannot emit more bits for the same price as fewer bits. READ_PORT charges $\text{bits} + S(\delta)$: the number of bits being read from the port, plus at least 1. More information in means more μ paid. The floor is still there; zero-cost certification remains impossible, and the cost tracks the actual bit content.

TENSOR_SET and TENSOR_GET manage per-module mu-tensor entries, tracking cost-tensor relationships between modules for the physics bridge (Section 16). The kernel tensor is a flattened 4×4 vector. The hardware has dedicated tensor state, and the closeout proofs now cover both TENSOR opcodes unconditionally in `GraphReconstructionBridge.v` and `RTLGapRegistry.v`. Bad tensor indices latch error in both semantics; good indices agree step-for-step.

9.6 Group 6: Witness and certification (3 opcodes)

These opcodes interact with the CHSH witness counters and the top-level certification flag.

CHSH_TRIAL(x, y, a, b, δ): record one CHSH trial.
Updates one of the eight witness counters based on measurement settings ($x \in \{0,1\}$, $y \in \{0,1\}$) and outcomes ($a \in \{0,1\}$, $b \in \{0,1\}$). The Coq constructor order is settings first, then outcomes. The counter for the setting pair (x, y) and same/different outcome bucket goes up by 1. Charges δ to μ.

This is a Tier 0 operation: it records data but does not activate any certification channel. It is provably free in the sense that CHSH_TRIAL does not and cannot trigger a cert-insight event (proven in `WitnessInsightGeneral.v`, `chsh_trial_is_tier0`).

CERTIFY(δ): top-level proof-carrying certification.
Sets `vm_certified = true`, charges $S(\delta) \geq 1$ to μ, advances PC. This is the simplest cert-channel activation: you are saying "I, the program, certify that this execution has reached an

attested state." The flag is set. The cost is charged. There is no way to set `vm_certified = true` without this opcode, and this opcode always charges ≥ 1.

REVEAL(m, *bits*, *cert*, δ): disclose a value.
Reveals *bits* bits from module m as part of a non-local revelation protocol, with certificate string *cert*. REVEAL charges bits $+ S(\delta)$: the number of bits being disclosed, plus at least 1. The cost is also recorded in the μ-tensor at the direction index encoded in the instruction, marking *where* in metric-space the revelation cost lands. Revealing 0 bits still costs at least 1. Revealing 32 bits costs at least 33. You cannot disclose more for the same price as less. This is what makes the CHSH revelation requirement concrete: obtaining non-local statistics requires explicit revelation, and that revelation is priced by how much you reveal.

MORPH_ASSERT (see Group 7): it activates the cert channel and belongs conceptually here too, but is listed in the morphism group because it operates on a morphism ID.

9.7 Group 7: Categorical morphism operations (7 opcodes)

These are the opcodes that make the Thiele Machine something a Turing machine cannot simulate without losing structure. I will explain what a morphism is before listing the opcodes.

A *morphism* is a record saying: "module A is related to module B in this specific, named way." Not just "A and B exist." Not just "A and B are connected by an edge." A morphism has a source module ID, a target module ID, coupling data (a list of (source-cell, target-cell) pairs plus a label string), and an is-identity flag.

The coupling data is actual relational content. MORPH creates a morphism with an empty coupling list initially. COMPOSE computes the relational composition of two morphisms' coupling lists: pair (a, c) is included if and only if there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct in `CategoryBridge.v`. MORPH_TENSOR concatenates two coupling lists. The coupling is not a programmer tag. It is data the machine builds and proves properties about. You can read back the number of coupling pairs via MORPH_GET (selector 2).

Why does this matter? Because two programs can have identical classical state (same registers, same memory, same μ) while having completely different morphism maps. A classical machine cannot distinguish them. The Thiele Machine can, via a single morphism probe. This is the Categorical Separation Theorem (Section 11).

Opcode	Effect	Cost
`MORPH`(r_d, s, t, c, δ)	If source module s and target module t exist, create a fresh morphism from s to t and write its generated ID into r_d. The current step uses empty coupling data; c is carried but not used by the kernel step.	δ
`COMPOSE`(r_d, m_1, m_2, δ)	Compose morphism $m_1 : A \to B$ with morphism $m_2 : B \to C$, create a fresh composed morphism, and write its ID into r_d. Fails if composition is undefined.	δ
`MORPH_ID`(r_d, m, δ)	Create identity morphism on module m and write its generated ID into r_d.	δ
`MORPH_DELETE`(id, δ)	Delete morphism id from the morphism map. Sets err if id does not exist.	δ
`MORPH_ASSERT`(id, *prop*, *cert*, δ)	Assert that morphism id exists, with property string *prop* and certificate string *cert*. If it does: sets `csr_cert_addr` to nonzero (activates Tier 1 cert channel). If it does not: sets err. EITHER WAY: charges $S(\delta) \geq 1$.	$S(\delta) \geq 1$
`MORPH_TENSOR`(r_d, m_1, m_2, δ)	Form the tensor product of morphisms m_1 and m_2 into a fresh morphism and write its ID into r_d.	δ
`MORPH_GET`(r, id, *field*, δ)	Read field (`source` / `target` / `coupling` / `is_identity`) of morphism id into register r.	δ

MORPH_ASSERT deserves special attention. It charges $S(\delta) \geq 1$ regardless of whether the assertion succeeds. This is the sharpest instance of No Free Insight: the *attempt to claim structural knowledge costs something*, even when the claim is false. A failed assertion is not free. A machine without a structural ledger has no way to express this constraint. In the Thiele Machine it is enforced by the step semantics and proven in `AbstractNoFI.v`.

9.8 Why 46 opcodes?

The number 46 comes from what the machine needs to do:

- Compute arithmetic and data: 14 ops (LOAD_IMM, XFER, ADD, SUB, MUL, AND, OR, SHL, SHR, LUI, XOR_LOAD, XOR_ADD, XOR_SWAP, XOR_RANK)
- Memory and control flow: 8 ops (LOAD, STORE, JUMP, JNEZ, CALL, RET, HALT, CHECKPOINT)
- I/O, heap, and tensor: 7 ops (READ_PORT, WRITE_PORT, EMIT, HEAP_LOAD, HEAP_STORE, TENSOR_SET, TENSOR_GET)
- Partition structure: 4 ops (PNEW, PSPLIT, PMERGE, PDISCOVER)
- Logic and certification: 3 ops (LASSERT, LJOIN, MDLACC)
- Witness and certification flags: 3 ops (CHSH_TRIAL, CERTIFY, REVEAL)
- Categorical morphisms: 7 ops (MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_TENSOR, MORPH_GET)

$14 + 8 + 7 + 4 + 3 + 3 + 7 = 46$.

Every opcode exists for a reason. None are redundant. If you removed CERTIFY, you could not activate `vm_certified`. If you removed MORPH, you could not build morphism relationships. If you removed CHSH_TRIAL, you could not track non-local correlations. The ISA is designed, not grown.

10 What a categorical morphism is: the category theory, from scratch

I said earlier that a morphism is a record saying "module A is related to module B in a specific way." That is informal. Let me be precise. But I am going to explain category theory from zero, because I did not know what it was when I started this project.

10.1 The idea of a category

A category is a collection of two things: objects and arrows between them.

The objects can be anything. In abstract math, they might be sets, or vector spaces, or topological spaces. In the Thiele Machine, the objects are modules (partitions of state).

The arrows are also called morphisms. An arrow goes from one object to another: $f : A \to B$ means "there is an arrow f from object A to object B." The arrow does not have to be a function. In the Thiele Machine, a morphism just says "there is a relationship of this type between these two modules."

Two laws must hold:

1. **Composition**: If $f : A \to B$ and $g : B \to C$, there must exist a composite $g \circ f : A \to C$. This is what COMPOSE implements.

2. **Identity**: For every object A, there must exist an identity arrow $\mathrm{id}_A : A \to A$. This is what MORPH_ID implements.

And composition must be associative: $(h \circ g) \circ f = h \circ (g \circ f)$.

Why composition? Because you want to chain relationships. If A relates to B and B relates to C, you want to say A therefore relates to C, transitively, without having to name every possible A-to-C path individually. Without composition, the structure collapses into a flat list of unconnected pairs. It's entirely useless for multi-hop reasoning.

Why identity? Because every object needs a minimal self-relationship. An isolated module with no connections to anything else still needs to be expressible in the framework. MORPH_ID is how you say "module A relates to itself with no coupling content." Without it, isolated objects fall outside the framework entirely, and any module with zero external relationships becomes unrepresentable.

Why associativity? Because the parenthesization of a chain should not matter. Whether you write $(h \circ g) \circ f$ or $h \circ (g \circ f)$, both mean the same three-hop path $A \to B \to C \to D$. If associativity failed, two programs that build the same chain in different groupings would produce different morphisms. That is a bug: the chain is the thing, not how you assembled it.

These laws are proven in Coq in `CategoryLaws.v`. Zero Admitted. Concretely: morphisms carry coupling data, a list of (source-cell, target-cell) pairs plus a label. COMPOSE produces a new morphism whose coupling pairs are the relational composition of the two inputs. This is proven in `CategoryBridge.v` (`graph_compose_morphisms_coupling`). The Thiele Machine's categorical layer is not just called a category; it satisfies the actual axioms on the actual coupling data.

10.2 Why modules as objects

In the Thiele Machine, the objects of the category are the modules of the partition graph. A module is a named piece of the computation: it owns a region of memory or state, it may carry certified axioms about that region, and it contributes a local μ cost.

A morphism between two modules says: "there is a named relationship between module A and module B, with explicit coupling data." The coupling data is a list of (source-cell, target-cell)

pairs. COMPOSE computes the relational composition of two couplings: pair (a, c) is in the composed coupling exactly when there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct. The machine tracks it. Via MORPH_GET (selector 2), you can read back the number of coupling pairs in any morphism.

What the coupling pairs represent is up to the programmer. They could encode:

- Module B refines module A (which cells in A map to which cells in B)
- Module A entangles with module B (which cell-pairs are correlated)
- Module A simulates module B (which states correspond)

The machine enforces the relational composition correctly. The interpretation is yours.

10.3 Why this is not just "extra state"

Here is the question I asked myself for a long time: if you can just add extra registers to a classical machine to store morphism IDs, why is the categorical layer special?

The answer is in the MORPH_ASSERT opcode and in the Categorical Separation Theorem.

Extra registers do not have provenance. You can write anything into a register. You can write 42 into register 5 and claim it is a morphism ID. But the Thiele Machine checks: did a MORPH instruction actually run that created morphism 42 with the source and target you claim? If not, MORPH_ASSERT fails. And MORPH_ASSERT costs $S(\delta) \geq 1$ even if it fails.

Building the morphism chain by calling MORPH, COMPOSE, and MORPH_ID is free when those instructions carry $\delta = 0$. Those ops don't have a mandatory positive floor. What costs is MORPH_ASSERT: the act of asserting that the morphism exists and activating the cert channel. That always costs $S(\delta) \geq 1$. You can build the chain for free, but you cannot certify it for free.

This is why the morphism layer is not "extra state." It is *provenance-bearing state*. The machine knows where it came from. A classical machine cannot fake this at the same cost. That is the Categorical Separation Theorem.

11 Categorical separation: the machine is strictly richer

Theorem 15 (Categorical Separation, `PartitionSeparation.v`, zero Admitted)**.** *There exist states s_1 and s_2 such that:*

- $s_1.$`vm_regs` $= s_2.$`vm_regs` *(identical registers)*
- $s_1.$`vm_mem` $= s_2.$`vm_mem` *(identical memory)*
- $s_1.$`vm_mu` $= s_2.$`vm_mu` *(identical μ)*
- $s_1.$`vm_err` $= s_2.$`vm_err` *(identical error flag)*
- $s_1.$`vm_pc` $= s_2.$`vm_pc` *(identical program counter)*
- $s_1.$`vm_certified` $= s_2.$`vm_certified` *(identical top-level certification flag)*

yet their morphism graphs differ. In the concrete witness, one state has an identity morphism in `pg_morphisms`*; the other has no morphisms.*

Construction. Build s_1 with `pg_morphisms = [(0, id)]`. Build s_2 with `pg_morphisms = []`. Set every classical field the same: registers, memory, μ, PC, error flag, and `vm_certified`. The equality proof is by reflexivity on those fields. The graph distinction is by list constructor mismatch: a one-element morphism list is not the empty list. □

Theorem 16 (Shadow Separation, `ShadowProjection.v`, zero Admitted)**.** *There exist two states with the same classical shadow but different morphism graphs, and a legitimate graph-preserving probe after which the morphism-level distinction remains.*

The probe used in `ShadowProjection.v` is deliberately boring: `ADD 0 0 0 0`. It is not a morphism operation. That is the point. The theorem is not relying on a failing probe to manufacture a difference. The difference is already in the structural state, and ordinary classical projection cannot see it.

Corollary 17 (Classical observer is blind, `ShadowProjection.v`, zero Admitted)**.** *No function of the classical shadow, the tuple (`vm_regs`, `vm_mem`, `vm_mu`, `vm_pc`, `vm_err`, `vm_certified`), can separate all Thiele states. The classical shadow is strictly lossy.*

Corollary 18 (Thiele strictly extends classical, `TuringStrictness.v`, zero Admitted)**.** *Every classical program leaves `pg_next_id` unchanged. There exist Thiele programs that change `pg_next_id` (via PNEW). Therefore Thiele strictly extends classical computation: not just in the sense that it can compute the same things, but in the sense that it can represent and probe states that classical programs cannot reach.*

The classical machine simulates inside the Thiele Machine faithfully (every classical program runs correctly, no side effects on the structural layer). But the Thiele Machine has programs that escape the classical fragment. This is proven, not claimed. Zero Admitted.

12 The knowledge receipt: what makes certification unforgeable

Let me show why the μ ledger is unforgeable by walking through a concrete four-act demonstration.

Act 1: The attempt costs, even when it fails.

Run MORPH_ASSERT(k) where morphism k does not exist. What happens:

- `vm_err` becomes true (the assertion failed).
- μ increases by $S(\delta) \geq 1$ where δ is the instruction's encoded cost parameter (the attempt cost something).

The machine paid for a failed assertion. A machine without a structural ledger has no way to express this. In the Thiele Machine, the cost of claiming knowledge is charged whether or not the claim is true.

Act 2: Build the chain.

Run this sequence:

$$\text{PNEW}(A);\ \text{PNEW}(B);\ \text{PNEW}(C);$$
$$\text{MORPH}(f, A, B);\ \text{MORPH}(g, B, C);\ \text{COMPOSE}(h, f, g)$$

Then MORPH_GET(r_0, h, source) and MORPH_GET(r_1, h, target). The resulting state has $r_0 = A$, $r_1 = C$, and morphism $h : A \to C$ exists in the morphism map. With $\delta = 0$ on each build step, $\mu = 0$. The morphism exists, but no structural cost has been paid yet.

Act 3: Certify.

Run MORPH_ASSERT(h). Morphism h exists. The assertion succeeds. `csr_cert_addr` goes nonzero. Tier 1 cert channel activated. μ goes up by $S(\delta) \geq 1$.

The machine now holds a μ-ledger entry saying: at this point in the trace, a valid composed morphism $A \to C$ existed, and the machine paid to confirm it. That entry is the knowledge receipt. It is in the ledger. It cannot be removed. It cannot be backdated. μ never goes down.

Act 4: Classical programs cannot forge it.

Program B takes a shortcut: it loads $r_0 = A$ and $r_1 = C$ via LOAD_IMM. Classical state is identical: same registers, same μ (both zero). No morphism h was ever created.

Apply MORPH_DELETE(h): Program A (Act 2) succeeds. Program B fails and sets err.

The μ ledger is not a counter that measures how much work was done. It is a record of which structural commitments were made. Program B did the same amount of "work" (in the μ sense) but did different work. The machine knows the difference.

13 How the machine is built: three layers

The Thiele Machine exists in three forms. They are intended to implement the same semantics. The three layers are tied together by refinement proofs and automated tests.

13.1 Layer 1: The Coq kernel

This is the mathematical ground truth. The Coq kernel defines:

- The VMState record (all 12 fields, as in Section 8)
- The vm_instruction type (all 46 opcodes as Coq inductive constructors)
- The vm_apply function: takes a state and instruction, returns a state. Total. Always returns something.
- All the theorems: NoFI, μ-monotonicity, μ-initiality, categorical separation, Turing strictness, etc.

The Coq kernel operates mostly over natural numbers, but the machine state is not an unbounded fantasy object. It fixes 32 registers and 65536 memory words, and its accessors wrap indices through those sizes. Register and memory writes are truncated by the kernel's 64-bit word helpers. μ itself is a natural number and can grow without a fixed machine-word ceiling in the kernel. The RTL target then instantiates a finite 32-bit datapath and bounded tables.

Zero Admitted means: no step in any theorem proof was asserted without derivation. The machine checked everything. I did not.

13.2 Layer 2: The OCaml extracted runner

Coq can automatically extract a Coq development to OCaml. The extracted runner executes all 46 opcodes from the Coq semantics, including the morphism operations.

The trust boundary here is named explicitly. `OCamlExtractionBridge.v` proves formal facts about the Coq-side observable `shadow_to_eo (vm_apply s i)`: exact μ accounting, nondecreasing μ, and totality. It also names the external runner boundary. The current file does not contain a real unproved Coq `Hypothesis` called `ocaml_runner_agrees`; the lemma with that name is a reflexive Coq-side witness. The actual cross-language claim, that the built OCaml binary returns the same observable as the Coq semantics, is checked by the parity test suite. I do not count that as a kernel theorem. It is an audited extraction boundary backed by tests.

13.3 Layer 3: The Kami RTL hardware

The Kami framework lets you write hardware specifications in Coq and extract them to Bluespec SystemVerilog. Bluespec SystemVerilog is a high-level hardware description language: you describe what the hardware should compute, including the registers, the rules, and the conditions under which state transitions fire. The Bluespec compiler generates standard Verilog RTL. Verilog RTL is the low-level gate-and-register description that synthesis tools can turn into actual

logic gates on an FPGA or ASIC. The Thiele Machine hardware is defined in `ThieleCPUCore.v` (Kami module) and extracted through this pipeline.

The hardware has:

- 32 registers, each 32 bits wide (data registers)
- 128-bit instruction encoding (`InstrSz = 128`)
- 65536 words of data memory
- bounded instruction transport and execution state
- 64 partition module slots (`PTableSz = 64`)
- 64 morphism descriptor slots (`MorphTableSz = 64`)
- An on-chip LASSERT FSM for formula verification
- A μ-ALU running in parallel with main execution

All 46 opcodes are implemented in the RTL design: `ThieleCPUCore.v` has hardware logic for all of them. The question is not which opcodes are in hardware, but which opcodes have a completed formal *bisimulation proof* showing the hardware circuit and the Coq spec agree step-for-step.

Formal bisimulation proofs exist for all 46 opcodes (zero Admitted, all Qed). Specifically:

- 36 opcodes have unconditional proofs. Thirty are covered through `SupportedOpcode`; CALL, RET, and CHSH_TRIAL are covered in `EmbedStep_WF.v`; TENSOR_SET, TENSOR_GET, and LASSERT are covered in `GraphReconstructionBridge.v`.
- 10 opcodes have conditional proofs (all Qed) under structural invariants that are proved inductive. They hold at initialization and are preserved by every instruction: PNEW, PSPLIT, PMERGE, and the seven morphism opcodes under `extended_hw_invariant` / `coupling_wf`.

Zero opcodes have unresolved coverage gaps. `RTLGapRegistry.v` records an empty gap registry (`rtl_gap_count = 0`, proven in Coq). The partition $36 + 10 + 0 = 46$ is itself a Coq theorem (`rtl_coverage_partition`).

The 46/46 proof coverage (all Qed, zero Admitted) is the correct relationship between an abstract ISA and its physical instantiation. The abstract model defines what is correct; the hardware implements all of it; the formal bisimulation proofs have been completed for all 46 opcodes. This is not a deficiency. It is honest proof accounting.

The abstract Thiele Machine is the Coq kernel. The hardware is a finite physical instantiation with explicit table sizes and word widths. The proofs state the bridge between them under the finite representation invariants where those invariants are needed. This is the only honest relationship physical hardware can have with an abstract mathematical model.

14 CHSH: when correlations break classical explanation

The Thiele Machine has opcodes for recording CHSH trials. I need to explain what CHSH is, and I want to build it from the ground up because the inequality itself is not automatic until you see where it comes from.

14.1 The Bell test setup, from scratch

Imagine two players, Alice and Bob, in separate rooms. They cannot communicate once the game starts. A referee sends Alice one of two questions ($a \in \{0, 1\}$). The referee sends Bob one of two questions ($b \in \{0, 1\}$). Alice answers $x \in \{0, 1\}$. Bob answers $y \in \{0, 1\}$.

The game rule: they want $x \oplus y = a \wedge b$. In plain language: their answers should be different if and only if both questions were 1. For all other question combinations, they should match.

Why 75% is the classical ceiling. Before the game, Alice and Bob can agree on any deterministic strategy: a function $x(a, \lambda)$ for Alice and $y(b, \lambda)$ for Bob, where λ is any shared randomness they agree on in advance. Once the game starts, no communication.

Fix any deterministic strategy and freeze λ. Then Alice has two predetermined answers, x_0 and x_1, and Bob has two predetermined answers, y_0 and y_1. Winning all four question combinations would require:

$$x_0 \oplus y_0 = 0, \quad x_0 \oplus y_1 = 0, \quad x_1 \oplus y_0 = 0, \quad x_1 \oplus y_1 = 1.$$

Now XOR the four left sides. Every variable appears twice, so the total is 0. XOR the four right sides and the total is 1. That is impossible. So every deterministic local strategy loses at least one of the four cases, and the best deterministic strategy wins at most 3 out of 4 question combinations.

Randomized strategies cannot do better: they are just mixtures of deterministic strategies, and averaging strategies that each win 75% at most gives you 75% at most. This is not a vague claim. It is a direct consequence of linearity of expectation.

So: maximum classical win rate = 3/4 = 75%. Proven by exhaustion in `ClassicalBound.v` (zero Admitted): an executable trace using only PNEW, PSPLIT, and CHSH_TRIAL with $\mu = 0$ achieves exactly this, and `MinorConstraints.v` proves no $\mu = 0$ program can exceed it.

14.2 The CHSH inequality: where it comes from

Now here is what took me a while to understand: why is there a specific algebraic combination S that detects whether correlations can be explained classically?

The key move is this. For any fixed shared randomness λ, Alice's output $A_a(\lambda) \in \{-1, +1\}$ (I'm using ± 1 encoding now, not 0/1) and Bob's $B_b(\lambda) \in \{-1, +1\}$. Consider the expression:

$$A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda)$$

Factor it:

$$= A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda))$$

Since B_0 and B_1 are each ± 1, either $B_0 + B_1 = \pm 2$ and $B_0 - B_1 = 0$, or $B_0 + B_1 = 0$ and $B_0 - B_1 = \pm 2$. The two terms cannot both be large. In either case, $|B_0 + B_1| + |B_0 - B_1| = 2$, and since $|A_0|, |A_1| \leq 1$, the entire expression is at most 2 in absolute value for any fixed λ.

Average over λ:

$$S = \langle A_0 B_0 \rangle + \langle A_0 B_1 \rangle + \langle A_1 B_0 \rangle - \langle A_1 B_1 \rangle$$

where $\langle A_a B_b \rangle$ is the correlation (expected value of the product) when questions are a and b. Since the expression before averaging was bounded by 2 for every λ, the average satisfies $|S| \leq 2$.

That is the argument. The combination with the minus sign is chosen exactly because the factoring trick works: one of the B-terms goes to zero and the other to ± 2, forcing the whole thing below 2. The inequality $|S| \leq 2$ is not a coincidence. It is a pure algebraic consequence of the outputs being ± 1 and being determined locally.

This is proven in Coq in `ClassicalBound.v` (zero Admitted). The formal statement: for any locally factorizable strategy, $|S| \leq 2$.

14.3 The quantum bound: why $2\sqrt{2}$?

Quantum mechanics allows $|S|$ to reach $2\sqrt{2} \approx 2.828$. Why that number?

In quantum mechanics, Alice's observable for question a is a Hermitian operator $\hat{A}_a$ on a Hilbert space, and similarly for Bob. For a shared quantum state ρ, the correlation is $\langle A_a B_b \rangle = \mathrm{tr}(\rho \cdot \hat{A}_a \otimes \hat{B}_b)$. Now form the CHSH operator:

$$\hat{S} = \hat{A}_0 \otimes \hat{B}_0 + \hat{A}_0 \otimes \hat{B}_1 + \hat{A}_1 \otimes \hat{B}_0 - \hat{A}_1 \otimes \hat{B}_1$$

The maximum of $|\langle \hat{S} \rangle|$ over all quantum states is the operator norm $\|\hat{S}\|$. For qubit observables with eigenvalues ± 1, this norm reaches exactly $2\sqrt{2}$ when the observables are chosen at the right angles on the Bloch sphere. The Bloch sphere is a way to represent any qubit state geometrically: a qubit $\alpha|0\rangle + \beta|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$ corresponds to a point on a unit sphere, with the two classical states $|0\rangle$ and $|1\rangle$ at opposite poles. A Hermitian observable with eigenvalues ± 1 corresponds to an axis through the sphere. The angle between two observables is the angle between their axes. The optimal CHSH setting places Alice's axes 45 degrees apart and Bob's axes 45 degrees from Alice's, giving a total separation in the operator algebra that drives the maximum eigenvalue of $\hat{S}^2$ to 8. You get $2\sqrt{2}$ from the Pythagorean theorem applied to a 2-dimensional operator algebra: the maximum eigenvalue of $\hat{S}^2$ is 8, so $\|\hat{S}\| = \sqrt{8} = 2\sqrt{2}$.

The repository proves this in a deliberately scoped way. `TsirelsonQuantumModel.v` proves that traces satisfying the repository's `quantum_realizable` zero-marginal NPA model obey the bound, in the form `c4_direct_tsirelson_abs_from_quantum_realizable`: $|S| \leq \sqrt{8}$. `TsirelsonUpperBound.v` supplies the rational threshold bookkeeping and the algebraic/classical side of the story. I am not claiming that `TsirelsonUpperBound.v` alone proves every possible quantum strategy is bounded. The formal claim is the named quantum-model theorem, and it has the `quantum_realizable` premise.

The gap between 2 and $2\sqrt{2}$ is real. No locally factorizable strategy can bridge it. That is the signature of quantum non-locality: correlations that cannot be produced by any shared classical randomness, no matter how much pre-game coordination you allow.

The Inquisitor's note on the rational approximation. `TsirelsonUpperBound.v` names the rational threshold $5657/2000 \approx 2.8285$ for exact rational comparisons. The real-valued quantum-model bridge uses $\sqrt{8}$ directly. Those are different proof surfaces, and the manuscript should not blur them.

14.4 What the Thiele Machine proves

Theorem 19 (Classical CHSH Bound, `ClassicalBound.v`, zero Admitted)**.** *For any locally factorizable strategy (Alice's answer depends only on a and λ; Bob's answer depends only on b and λ; λ is shared randomness), $|S| \leq 2$.*

Theorem 20 (Non-locality of CHSH violation, `CHSHStatisticalBridge.v`, zero Admitted)**.** *WitnessCount configurations with $S > 2$ cannot be produced by any local hidden variable model.*

Theorem 21 (Tsirelson upper bound for the repository quantum model, `TsirelsonQuantumModel.v`, zero Admitted)**.** *If a trace is quantum-realizable in the repository's zero-marginal NPA model, then its CHSH value satisfies $|S| \leq \sqrt{8} = 2\sqrt{2}$.*

What does this have to do with computation? The connection is through the witness counters and the certification layer. If a Thiele Machine program accumulates CHSH trial results (via CHSH_TRIAL) and then certifies that $S > 2$ (via CERTIFY or LASSERT), the certification is non-free. Certifying non-local statistics requires paying $\mu \geq 1$. The statistics themselves (raw CHSH trials) are free. Claiming you know what they mean is not. This is the witness insight taxonomy (Section 5).

14.5 The REVEAL requirement

Getting $S > 2$ in any computational context requires revelation. Here is why that is not just a policy: it is a structural fact about the step semantics.

Each CHSH_TRIAL records setting bits and outcome bits into witness counters. The cross-correlations $\langle A_a B_b \rangle$ cannot be justified without some explicit certification/revelation path that combines or attests to the relevant data.

This is proven in `RevelationRequirement.v` (zero Admitted) in the repository's operational vocabulary: if execution starts with `csr_cert_addr = 0` and ends with `csr_cert_addr` $\neq$ `0`, then the trace used a revelation-class instruction. The theorem's disjunction includes `REVEAL`, `EMIT`, `LJOIN`, `LASSERT`, and `MORPH_ASSERT`; runtime policy treats `REVEAL` as the primary partition-disclosure path. Trying to get certified non-local statistics above the classical bound without paying one of those costs is structurally impossible.

15 Landauer: information erasure costs energy

Rolf Landauer proved in 1961 that erasing one bit of information from a physical system must release at least $kT \ln 2$ joules of heat into the environment, where k is Boltzmann's constant and T is the temperature of the environment. I want to explain where that formula comes from because it matters for how the μ ledger connects to physics.

15.1 Where the formula comes from

A bit of information is stored in a physical system with two distinguishable states: call them 0 and 1. The system might be a tiny magnet pointing up or down, a capacitor charged or uncharged, a transistor on or off.

"Erasing" the bit means forcing the system to state 0 regardless of what it was before. Before erasure, the bit could be 0 or 1; you have no idea which. After erasure, you know it is 0. You have gone from maximum uncertainty (1 bit of entropy) to zero uncertainty.

Here is the heat physics. Total entropy of the universe must not decrease (second law of thermodynamics). The bit's entropy went down by $k \ln 2$ (that is what "1 bit = $\ln 2$ nats of entropy" means in physical units). Some other part of the universe must gain at least that much entropy to compensate. The cheapest way to dump entropy is to release it as heat into a heat bath at temperature T. Dumping $\Delta S = k \ln 2$ of entropy into a bath at temperature T requires releasing heat $Q = T \cdot \Delta S = kT \ln 2$.

That is Landauer's argument. It is not complicated. Information is physical. The bit has to go somewhere. Destroying it means the physical system's entropy decreases, and the second law proves you pay for that in heat.

I found this argument important because it is the same kind of argument as No Free Insight: you cannot get structure for free, and the machine tracks where you paid. The difference is that Landauer's principle is about physical energy and the μ ledger is a formal accounting tool. The connection between the two is real but conditional.

15.2 What the Thiele Machine proves

Theorem 22 (μ-Landauer Step Bound, `LandauerDerivation.v`, zero Admitted)**.** *For any instruction i, the repository's conservative `irreversible_bits i` indicator is bounded by `instruction_cost i`. For a single step, the μ increase is at least `irreversible_bits i`; for a trace, total irreversible-bit indicators are bounded by total instruction cost.*

This is a theorem about the formal step semantics of the machine. It says the machine's cost accounting is Landauer-shaped: every positive-cost irreversible indicator used by this file is

covered by μ. It is deliberately coarse. It is not a microscopic physical erasure model for every possible hardware transition.

The physical interpretation, that μ costs correspond to actual thermodynamic entropy, is conditional. It depends on a named hypothesis (`mu_landauer_unruh_calibrated`) that calibrates μ to physical energy units. That hypothesis is physically motivated and consistent with the formal development, but it is not proven in Coq. It is a bridge to physics, not a derived fact. I say this directly because it is true.

The formal content of `LandauerDerivation.v` is narrower and cleaner: (1) only CERTIFY sets `vm_certified := true`, proven by case analysis on every instruction; (2) CERTIFY's cost is $S(\delta) \geq 1$ by construction; (3) therefore certification always requires positive μ-cost; (4) total μ-cost bounds the file's `total_irreversible_bits` measure over the trace. None of this requires a physical calibration. Physical calibration would plug in the Boltzmann constant and the temperature, turning μ counts into joules. That step is the named bridge hypothesis.

16 Einstein from computation: what we proved and what we did not

The Thiele Machine's μ costs define a geometry. Here is what that means, from first principles.

16.1 From partition data to a discrete geometry

Each module in the partition graph carries structural data, and several bridge files use μ-derived quantities as geometry-like weights. The discrete Gauss-Bonnet file is more specific than that: it reads the partition graph as a triangulated combinatorial object and uses an equilateral angle model where every triangle corner contributes $\pi/3$. It does not derive those angles directly from μ costs.

A discrete triangulation defines a curvature-like angle defect. To see why: on a flat surface, if you draw a triangle, the interior angles sum to 180°. On a curved surface, the angles can sum to more or less than 180°. Curvature is the amount by which triangles deviate from flat. In the equilateral discrete model, curvature at a vertex is the defect left after subtracting the incident triangle angles from 2π.

The Euler characteristic χ is a topological invariant of the simplicial complex: a number that depends on the overall shape of the space, not its geometry. For a triangulation with V vertices, E edges, and F faces: $\chi = V - E + F$. For a sphere, $\chi = 2$. For a torus, $\chi = 0$. The continuum Gauss-Bonnet theorem says the total curvature of a closed surface equals $2\pi\chi$: the geometry and the topology are linked.

16.2 What is proven

Theorem 23 (Discrete Gauss-Bonnet, `DiscreteGaussBonnet.v`, zero Admitted)**.** *For any* `well_formed_triangulated` *partition graph g in the repository's equilateral angle model,*

$$\texttt{total\_angle\_defect}\ g = 5\pi \cdot \texttt{euler\_characteristic}\ g.$$

This is a fully proven theorem about the repository's discrete triangulation model. It is not the continuum Gauss-Bonnet theorem and it is not a derivation of angles from μ cost. Zero Admitted.

The chain from this to Einstein's field equations goes through five steps:

1. Gauss-Bonnet (proven: `DiscreteGaussBonnet.v`)
2. Clausius entropy-area relation (proven: `ClausiusFromEntropyArea.v`)
3. Raychaudhuri equation (proven conditionally: `DiscreteRaychaudhuri.v`)

4. Jacobson derivation of Einstein equations from thermodynamics (chain assembled in `EinsteinEmergence.v`)

5. Full tensor Einstein equations $G_{\mu\nu} = \kappa T_{\mu\nu}$ (proven in section: `EinsteinEquationsFull.v`)

Let me explain steps 3 and 4 from first principles, because they carry the most hidden load.

The Raychaudhuri equation. Imagine two nearby light rays traveling through space. They start parallel. As they travel, they can converge (focus toward each other) or diverge (spread apart). The rate of focusing is called the "expansion" of the beam.

The Raychaudhuri equation says: the rate of change of the expansion depends on the Ricci curvature of the spacetime along the beam direction. Positive Ricci curvature means the beam converges. This is gravitational focusing: matter curves spacetime, spacetime curves geodesics, and geodesics converge. This is how gravity works in General Relativity.

In the Thiele Machine, the discrete analog is: the null congruence expansion (how fast adjacent null geodesics of the μ-metric converge) decreases when the effective Ricci curvature is positive. `DiscreteRaychaudhuri.v` proves this for the μ-geometry. The result is conditional on a named hypothesis (`lorentzian_coupling_positive`) because the discrete geometry is Euclidean by construction and requires an explicit sign choice to be treated as Lorentzian. That sign choice is physically motivated but not proved from the Coq definitions alone.

The Jacobson derivation. Ted Jacobson showed in 1995 that if you apply the first law of thermodynamics ($dE = T\,dS$) to the boundary of every small causal region of spacetime, and if entropy is proportional to the *area* of that boundary, then Einstein's field equations follow as a thermodynamic equation of state. Why area and not volume? Jacob Bekenstein argued in 1973 that the information in a region of space is bounded by its boundary area, not its volume; the interior degrees of freedom cannot exceed what can be encoded on the surface at one bit per Planck-area patch. Stephen Hawking confirmed this for black holes: a black hole's entropy is exactly proportional to its event horizon area ($S = A/4G\hbar$ in natural units). This is surprising. Intuitively volume should hold more information than surface. The answer is that gravity compresses the interior: if you tried to pack more entropy into a region than its boundary allows, the region would collapse into a black hole. The surface is the ceiling. In the Thiele Machine's formal development, this is a named hypothesis (Bekenstein saturation): the claim that the partition graph's boundary area bounds its interior information content is taken as a bridge premise, not derived.

The key move: gravity is not fundamental. It is what you get when you treat spacetime as a thermodynamic system. The Einstein equations say that the geometry of spacetime adjusts so that entropy is extremized, just as pressure equilibrates in a gas. You do not need quantum mechanics or strings. You need thermodynamics applied to the geometry.

The Thiele Machine's μ costs play the role of entropy. The partition graph's morphism layer plays the role of the causal boundary. `JacobsonBridgeComponents.v` makes the two components explicit: a Clausius-type relation (first law for the μ-geometry horizon) and a Raychaudhuri-to-Einstein link (focusing implies curvature constraints). Together, they imply the Einstein target. `EinsteinEmergence.v` assembles the chain.

Theorem 24 (Full Tensor Einstein Equations, `EinsteinEquationsFull.v §10`, zero Admitted in section)**.** *For all* $\mu, \nu \in \{0, 1, 2, 3\}$*:*

$$G_{\mu\nu}(s, sc, v) = \kappa \cdot T_{\mu\nu}(s, sc, v)$$

16.3 What Theorem 24 actually says

Theorem 24 is conditional. It is proven inside a Coq `Section` with three named hypotheses:

- `diagonal_metric_h`: the μ-metric is diagonal (no off-diagonal metric terms).

- `diagonal_efe_h`: the diagonal Einstein equations hold (already proven separately).
- `off_diagonal_ricci_zero`: the off-diagonal Ricci curvature vanishes.

These are not `Admitted` proofs. They are explicitly named hypotheses that make the trust boundary auditable. The proof of Theorem 24 from these three hypotheses is zero Admitted: the derivation is valid. The hypotheses themselves are physically motivated (the off-diagonal Ricci vanishes in vacuum for symmetric metrics, and in the continuum limit of many discrete geometries) but not proven in Coq.

The honest claim: the Thiele Machine's μ geometry is consistent with Einstein's field equations in the diagonal metric case, and the extension to the full tensor case is proven conditional on the off-diagonal Ricci hypothesis. This is not a claim that gravity *is* computation. It is a claim that the formal analog is consistent.

If you want to know when the hypotheses fail: any situation where the μ-metric has strong off-diagonal correlations between modules would stress-test `off_diagonal_ricci_zero`. Designing an experiment to distinguish the μ-geometry from standard GR is an open problem. I have not solved it.

17 What Coq is, and why I used it

Coq is a proof assistant. It is a programming language where the type system is so expressive that you can write proofs as programs and have the compiler check whether they are valid.

Here is the idea. In a normal programming language, a type says something about what kind of value a variable holds. In Python, `x: int` means x is an integer. In Coq, types are propositions. The type `n > 0` means "the proof that n is positive." A function from `n > 0` to `n + 1 > 0` is a proof that if n is positive, $n + 1$ is too.

The compiler checks whether the types line up. If your proof has a hole and you write "trust me," the compiler rejects it. There is no way to convince the compiler of a false theorem by being persuasive or authoritative. The only way through is to actually prove it.

I used Coq because I do not trust informal arguments. Including my own. I spent months being convinced by my own reasoning that various claims were true, only to find when I tried to write the Coq proof that some key step was wrong. The machine caught it. That is the point.

17.1 The Inquisitor

Beyond Coq's type checker, the project has a custom proof auditor called the Inquisitor (`scripts/inquisitor.py`). It runs on every commit and checks:

- No `Admitted` anywhere in the active proof files
- No circular proofs (theorems that depend on themselves through the import graph)
- No vacuous theorems (theorems that prove things like `True` or `0 = 0` but are claimed to be content)
- No tautological implications (theorems of the form $P \to P$)
- No smuggled constraints (definitions that assume what they should be proving)

Current Inquisitor status: zero HIGH/MEDIUM/LOW findings across all 189 Coq files.

18 Zero Admitted: what it actually means

"Zero Admitted" means: every theorem in the proof tree was derived without any gaps. No step was asserted without a derivation. The machine verified the complete chain.

This is what it does NOT mean:

- It does not mean the theorems prove physical reality. The theorems prove things about the Thiele Machine model, which is a formal object in Coq. Whether the model correctly describes physical reality is a separate empirical question.
- It does not mean the Coq axioms themselves are consistent. Coq is based on the Calculus of Constructions, a formal language where types and programs are the same thing, so a proof is literally a program and a proposition is literally a type. Within this system, constructing a term of a given type means constructing a proof of the corresponding proposition. This foundation is believed to be consistent but this has not been proven within Coq itself (Gödel's incompleteness theorem prevents any sufficiently expressive system from proving its own consistency).
- It does not mean the hardware implements exactly the Coq semantics in all cases. All 46 opcodes have Qed bisimulation proofs (36 unconditional, 10 under inductive structural preconditions, zero gaps); documented in `RTLGapRegistry.v`.
- It does not mean the OCaml runner binary is proven inside Coq to be bitwise identical to the Coq semantics. `OCamlExtractionBridge.v` proves Coq-side observable facts and the parity suite checks the extracted binary empirically.

The kernel tier (Section 7) is the strongest tier. The theorems listed there have zero Admitted and zero named hypotheses. They are machine-checked derivations from the Coq type theory axioms and the Thiele Machine definitions. Nothing else.

The bridge tier involves named hypotheses and documented trust boundaries, including `mu_landauer_unruh_calibrated`, `off_diagonal_ricci_zero`, and the external OCaml parity boundary. These are explicitly labeled and auditable. They are not Admitted proofs. They are documented limits of the formal claim, which is how honest formal development works.

19 How to break this

Every claim in this document has a falsification condition. Here are the main ones.

Falsify No Free Insight.
Find a Thiele Machine program trace that:

- starts with `vm_certified = false` and `vm_mu = 0`
- ends with `vm_certified = true`
- ends with `vm_mu = 0`

If you find such a trace, the Coq proof of `certification_requires_positive_mu` is wrong and the compilation will fail when you add the counterexample as a theorem.

Falsify μ-monotonicity.
Find a state s and instruction i such that (`vm_apply` $s\ i$).$\mu < s.\mu$. The Coq proof of `vm_apply_mu` will fail.

Falsify categorical separation.
Prove that for ALL states s_1, s_2 with identical classical shadow, ALL instructions produce the same error behavior. This would contradict `categorical_separation` in Coq, meaning the proof has an error.

Falsify Turing strictness.
Prove that every Thiele program can be simulated by a classical program with the same observable

behavior (registers, memory, μ) without ever touching the partition graph or morphism map. This would contradict `D5_thiele_strictly_extends_classical`.

Falsify structural entitlement.
Find two Thiele states with the same classical shadow where no Thiele-level probe or state predicate can ever distinguish their structural fields. That would contradict `shadow_strictly_lossy`. Or produce a strict feasible-set subset with a distinguishing observation that does not yield strictly stronger membership predicates; that would contradict `feasible_strict_subset_implies_strict_predicates`.

Falsify universal certification cost.
Build a `CertificationSystem` satisfying the one-step rule `cs_cert_costs`, then give a trace from uncertified to certified with total cost 0. That would contradict `universal_nfi_any_substrate`. If you instead drop the one-step rule, you have found a free-certification system, not a counterexample.

Falsify the classical CHSH bound.
Exhibit a locally factorizable strategy that achieves $|S| > 2$. This would contradict `chsh_stat_violation_not_local` and also violate known mathematics.

Stress-test the physics bridge.
Design an experiment where a physical system implementing the Thiele Machine's μ-geometry produces off-diagonal Ricci curvature. If `off_diagonal_ricci_zero` fails empirically, the conditional Einstein theorem `full_tensor_efe_conditional` loses its grounding. The theorem itself would still be valid (it is still true that the hypothesis implies the conclusion), but the hypothesis would be known false.

The Inquisitor standard.
Run `python scripts/inquisitor.py` on the repository. If it finds any HIGH/MEDIUM/LOW finding, the proof hygiene has degraded. Current status: zero findings. If that changes, it is a bug.

20 The full claim ledger

These are the kernel-tier claims (zero Admitted, zero named hypotheses):

1. **no_free_certification**: csr_cert_addr going $0 \to$ nonzero in one step requires cost ≥ 1.
2. **no_free_certification_mu**: same, with μ increment ≥ 1.
3. **no_free_certification_trace_mu**: trace-level version.
4. **no_free_certification_certified**: vm_certified going false$\to$true requires cost ≥ 1.
5. **certification_requires_positive_mu**: both channels covered.
6. **universal_nfi_any_substrate**: any abstract certification system satisfying the one-step cost rule has trace-level non-free certification.
7. **thiele_universal_nfi_cert_addr**: universal theorem instantiated for Thiele's `csr_cert_addr` channel.
8. **thiele_universal_nfi_certified**: universal theorem instantiated for Thiele's `vm_certified` channel.
9. **thiele_represents_simulating_cert_system**: any certification system with a simulation morphism into Thiele transfers certification into a Thiele certified execution.

10. **forget_kernel_is_eq_on_classical**: the classical snapshot quotient forgets exactly the non-classical fields.
11. **blindness_non_injective**: distinct Thiele states can have the same classical snapshot.
12. **certification_is_lost**: certification can be in the kernel of the classical forgetful map.
13. **shadow_strictly_lossy**: the classical shadow forgets morphism structure.
14. **thiele_nfi_pc_indexed**: PC-indexed execution version.
15. **mu_is_initial_monotone**: μ is the unique canonical cost measure.
16. **vm_apply_mu**: exact ledger identity $\mu' = \mu + \delta$.
17. **kernel_certified_implies_positive_mu**: from zero, reaching certified implies $\mu > 0$.
18. **feasible_strict_subset_implies_strict_predicates**: strict feasible-set narrowing plus a distinguishing observation yields a strictly stronger predicate.
19. **strengthening_requires_structure_addition**: certified predicate strengthening requires a structure-addition event.
20. **b4_information_reduction_derives_strict_predicates**: feasible-set reduction can generate the strict-predicate premise used by NoFI.
21. **info_priced_cert_executions_bound**: executed cert-setting events are bounded by $\Delta\mu$.
22. **info_priced_posterior_representative_reduction_bound**: with an explicit decision-tree/posterior-representative witness, feasible-set compression is bounded by $\Delta\mu$.
23. **structural_entitlement_representation**: strict narrowing plus distinguishability, certified posterior admissibility, and an explicit posterior-representative reduction imply a strictly stronger predicate, a structure-addition event, and a $\Delta\mu$ lower bound.
24. **every_sound_structural_shortcut_lands_here**: every packaged `SoundStructuralShortcut` instantiates the structural-entitlement representation theorem.
25. **categorical_separation**: classical-identical states distinguishable by morphism probe.
26. **classical_observer_cannot_separate**: no classical function separates morphism-distinct states.
27. **shadow_proj / shadow_separation_theorem**: formal surjection, strictly lossy.
28. **D2_faithfulness**: classical programs embedded faithfully in Thiele.
29. **D3_conservativity**: classical programs leave structural layer untouched.
30. **D4_strictness**: there exist Thiele steps that escape the classical fragment.
31. **D5_thiele_strictly_extends_classical**: Thiele strictly exceeds classical semantics.
32. **degenerate_projection_theorem**: shadow projection is the classical equivalence quotient.
33. **no_classical_certification_decider**: no classical machine can decide morphism-certification.

34. **chsh_stat_violation_not_local**: CHSH-violating statistics are non-local.

35. **structural_trace_preserves_cert_addr**: traces with no cert-address setter preserve `csr_cert_addr`.

36. **partition_refinement_nonfree**: certified partition knowledge cannot be free.

37. **partition_free_but_certification_nonfree**: partition structure can be built at $\delta = 0$, but certified partition knowledge cannot be acquired for free.

38. **witness_insight_nonfree_general**: certified non-local witness insight requires $\mu \geq 1$.

39. **witness_insight_complete_taxonomy**: full three-tier taxonomy proven.

Bridge-tier claims (zero Admitted, with named hypotheses or documented trust boundaries):

- **Hardware bisimulation**: 46/46 opcodes with Qed proofs (36 unconditional, 10 under inductive structural preconditions, 0 gaps); documented in `RTLGapRegistry.v` (`rtl_gap_count = 0`).
- **OCaml extraction**: `ocaml_bisimulation_closure` proves NoFI, μ-monotonicity, and totality for the Coq-side extracted observable. The external OCaml binary equivalence is a parity-test trust boundary, not a kernel theorem.
- **Structural advantage**: `StructuralAdvantage.v` proves exact μ costs and the arithmetic time-tax facts for the factored-search witness; executable tests check the measured VM behavior.
- **Cost formula forcing**: `cost_necessity` + `cost_uniqueness`, conditional on Landauer-Unruh calibration for physical interpretation.
- **Einstein equations**: the full tensor EFE theorem is conditional on the named off-diagonal Ricci hypothesis.

21 What I don't know

This document is honest about what is proven. Here is what I don't know.

Physical interpretation of μ. The formal development is consistent with Landauer's principle and with the Einstein equation structure. Whether the μ ledger actually corresponds to thermodynamic entropy in a physical Thiele Machine is an open empirical question. I think it does. I cannot prove it.

Full informal representation theorem for structural entitlement. The repository now proves `structural_entitlement_representation` and `every_sound_structural_shortcut_lands_here`. For the explicit witness class, structural entitlement becomes strict predicate strengthening, requires structure addition, and carries a $\Delta\mu$ lower bound. I have not proven the open translation theorem: whether every human informal story about symmetry, factorization, modularity, independence, or decomposition can always be packaged as a `SoundStructuralShortcut`. That is the next theorem layer. It is also the methodological demand: if an argument can be pushed down into explicit witnesses, then it has to be pushed to bedrock; if it can still be pushed, it has to be pushed again.

Complexity implications. The No Free Insight theorem says structural certification costs $\mu \geq 1$. `StructuralAdvantage.v` gives a concrete factored-search witness where paying constant μ changes the search shape from N^2 to $2N$, with exact cost arithmetic and executable tests. That is not a classical complexity-class separation. It does not resolve P vs NP. Formalizing classes such as μP and proving separations between them remains open.

Unconditional Shannon-style lower bound. `MuShannonBridge.v` proves quantitative bounds when the decision-tree, fiber, or posterior-representative witness is explicit. It also records that the naive deterministic single-trace entropy claim is false in general. The missing step is an expectation-level, whole-tree, or distributional semantics that turns arbitrary feasible-set reduction into the required witness.

Quantum mechanics connection. The CHSH and Tsirelson results are proven for the machine model. Whether a physical Thiele Machine can actually produce CHSH-violating statistics (exploiting quantum entanglement) depends on hardware design and quantum engineering that is outside the scope of this project.

Off-diagonal Ricci. The condition `off_diagonal_ricci_zero` is physically reasonable (it holds in vacuum for symmetric metrics) but I have not proven it in Coq for the general μ-geometry case. This is the main open gap in the physics bridge.

I could pretend these are answered. I could wave at them vaguely and say "future work." I don't know. These are real open questions.

22 Conclusion

That's the whole thing.

A Turing machine is blind to global structure. It can compute anything but it has no internal account of when a computation becomes entitled to use a decomposition, invariant, morphism, or narrowed feasible set. The Thiele Machine makes that entitlement visible, first-class, and cost-bearing. No Free Insight is the first conservation law: you cannot certify stronger claims than you started with without paying in a monotone ledger μ that records every structural commitment.

The proof story is stronger than a one-line ledger rule. Classical shadows are many-to-one quotients that lose entitlement-relevant structure. Strict feasible-set narrowing can be turned into strictly stronger predicates. Certified strengthening requires structure addition. Any abstract certification system satisfying the one-step cost rule has trace-level non-free certification. And the structural-entitlement representation theorem welds the explicit witness case together: narrowing, distinguishability, certification, structure addition, and the $\Delta\mu$ lower bound. Every packaged `SoundStructuralShortcut` lands in that theorem. That is the formal spine of the project.

The machine has 46 opcodes. Most of them are ordinary compute operations that carry zero structural cost. The mandatory positive-cost certification/revelation class is LASSERT, LJOIN, EMIT, REVEAL, READ_PORT, CERTIFY, and MORPH_ASSERT. Partition and morphism-building operations can construct structure at $\delta = 0$; the paid step is the checked assertion or certification that lets the trace claim it. That is the point of No Free Insight.

The categorical morphism layer is strictly richer than any classical equivalent. Two programs with identical registers, memory, μ, and error flag can differ in morphism structure, and no classical observer can see that difference. The Coq proof checks it.

The hardware design implements all 46 opcodes in RTL. Formal bisimulation proofs, Coq proofs that the hardware step function matches the Coq spec step-for-step, have been completed for all 46 of them (zero Admitted). 36 have unconditional proofs; 10 have proofs under inductive structural invariants (proved to hold at initialization and be preserved by every instruction). The gap registry is empty (`rtl_gap_count = 0`). This is documented in `RTLGapRegistry.v`.

The physics connections, Landauer, Einstein, CHSH, and Tsirelson, are real but carefully scoped. Some are proven inside the repository's formal models. Some are conditional. None are claimed to settle physical reality. They are formal analogs and bridges, and I have been honest about which is which.

Zero Admitted. Zero Inquisitor findings. Every claim has a falsification condition. If this is wrong, it will fail visibly, and that is exactly how it should be.

I'm a car salesman. I built this. Read the proofs.